Deepa Garg **Semester 05**

2401029 T12 **ASD&D Lab**

Experiment No. 05

Aim

To automate application deployment using Github/ Jenkins.

Lab Outcome: LO3

Theory

Continuous Integration and Continuous Deployment (CI/CD)

Continuous Integration (CI) is a software development practice in which developers frequently merge their code changes into a shared repository, after which an automated build and test process verifies each change. Continuous Deployment (CD) extends this practice by automatically releasing every change that passes the build and test stages into a production or staging environment. Together, CI/CD reduces manual effort, catches integration errors early, and enables faster and more reliable software delivery.

Jenkins

Jenkins is a widely used open-source automation server that helps automate the parts of software development related to building, testing, and deploying applications. It supports the creation of Pipelines, which define the complete workflow of an application, from source-code checkout to build, test, and deployment, as a sequence of stages. Jenkins can be integrated with version-control systems such as GitHub so that a pipeline is triggered automatically whenever new code is pushed.

Declarative Pipeline Syntax

Jenkins Pipelines can be written using Declarative Pipeline syntax, which provides a simplified and structured way of defining a pipeline using a Jenkinsfile. The core elements of this syntax are:

- pipeline { } — the outermost block that contains the entire pipeline definition.
- agent any — instructs Jenkins to run the pipeline on any available agent/node.
- stages { } — a container for one or more stage blocks executed sequentially.
- stage('name') { } — represents a single logical phase of the pipeline, such as Build, Test, or Deploy.
- steps { } — contains the actual commands or actions executed within a stage.

A minimal pipeline used in this experiment is shown below:

```
pipeline {
```

```
    agent any
    stages {
        stage('Build') {
            steps {
                echo 'Building the application...'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing the application...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying the application...'
            }
        }
    }
}
```

Meaning of Each Stage

Build — stage('Build')

Prepares or creates the application. For a Java application, this could involve compiling the source code; for a Python application, it could mean installing dependencies or packaging the application. For the simple HTML application used in this experiment, this stage represents the point at which the application would be prepared for testing.

Test — stage('Test')

Checks whether the application works correctly. In a full setup, automated test scripts would be executed here to validate functionality before deployment.

Deploy — stage('Deploy')

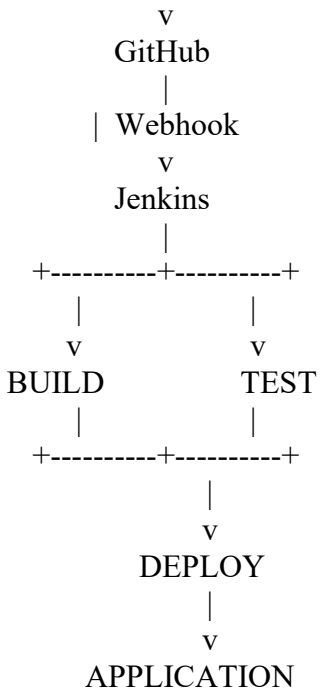
Places the successfully tested application into its target environment — for example, copying the HTML application to a deployment location once the Build and Test stages complete successfully.

Role of the GitHub Webhook

A GitHub webhook notifies Jenkins whenever a change (commit or push) occurs in the connected repository. Once the 'GitHub hook trigger for GITScm polling' option is enabled on the Jenkins job and the Pipeline is configured to fetch its script from the repository (Pipeline script from SCM), every push to the repository automatically triggers a new Jenkins build without any manual intervention. This is what converts a manually triggered pipeline into a true CI/CD workflow.

Overall CI/CD Flow

YOU
|
| Change Code



Implementation

Part 1 — Installing Jenkins

- Check that Java is installed on the machine by running: java --version
- Open the official Jenkins Windows installation page and download the Jenkins LTS Windows installer (.msi).
- Run the downloaded installer as Administrator (right-click → Run as administrator) and complete the installation.
- Press Windows + R, type services.msc, and confirm that the Jenkins service shows a status of Running.
- Open a browser and navigate to https://localhost:8080 (or the configured Jenkins port) to confirm Jenkins is up and access the Jenkins Dashboard.

Part 2 — Creating the First Pipeline Job

On the Jenkins Dashboard, click New Item, enter a name for the job (e.g., GitHub-Jenkins-Demo), select Pipeline as the item type, and click OK.

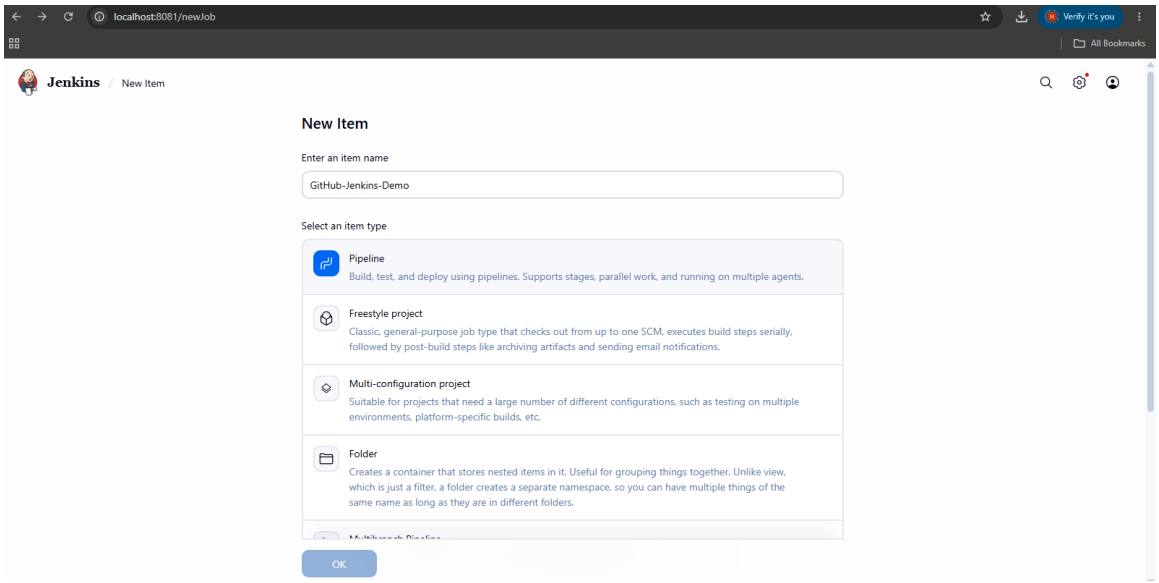


Fig 1: Creating a new Pipeline item named 'GitHub-Jenkins-Demo' in Jenkins

In the configuration page, scroll down to the Pipeline section and enter the Declarative Pipeline script (Build, Test, Deploy stages) directly in the Script box, then click Save.

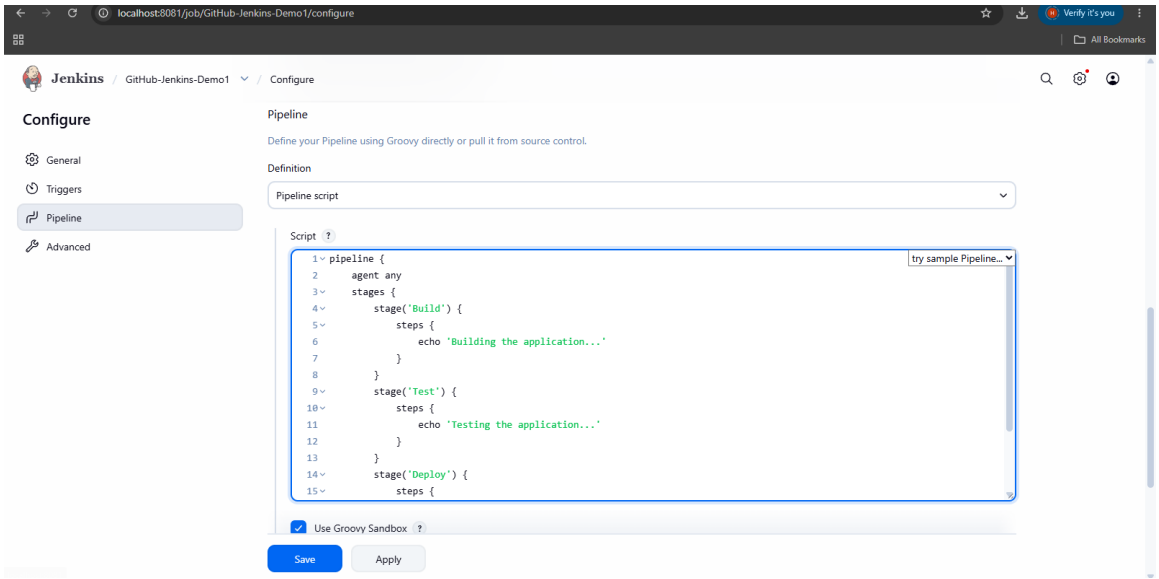


Fig 2: Entering the Declarative Pipeline script directly in the Jenkins Pipeline configuration

Part 3 — Running the Pipeline Locally

After saving, the job page for GitHub-Jenkins-Demo1 opens with no builds yet.

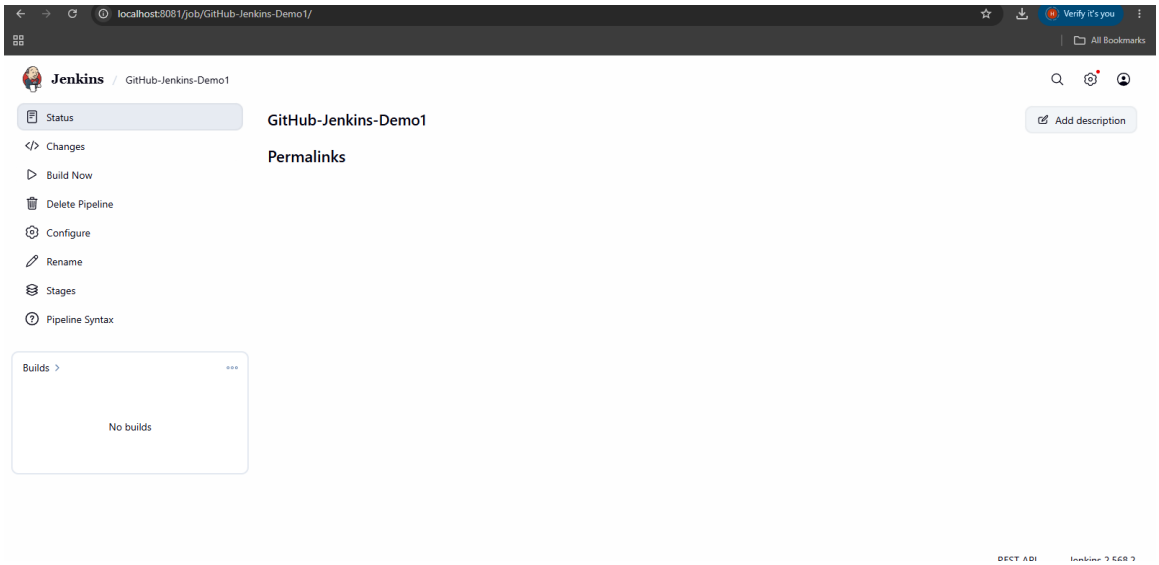


Fig 3: Job page showing 'No builds' before the first run

Clicking Build Now schedules a new build, confirmed by the 'Build scheduled' notification at the bottom of the page.

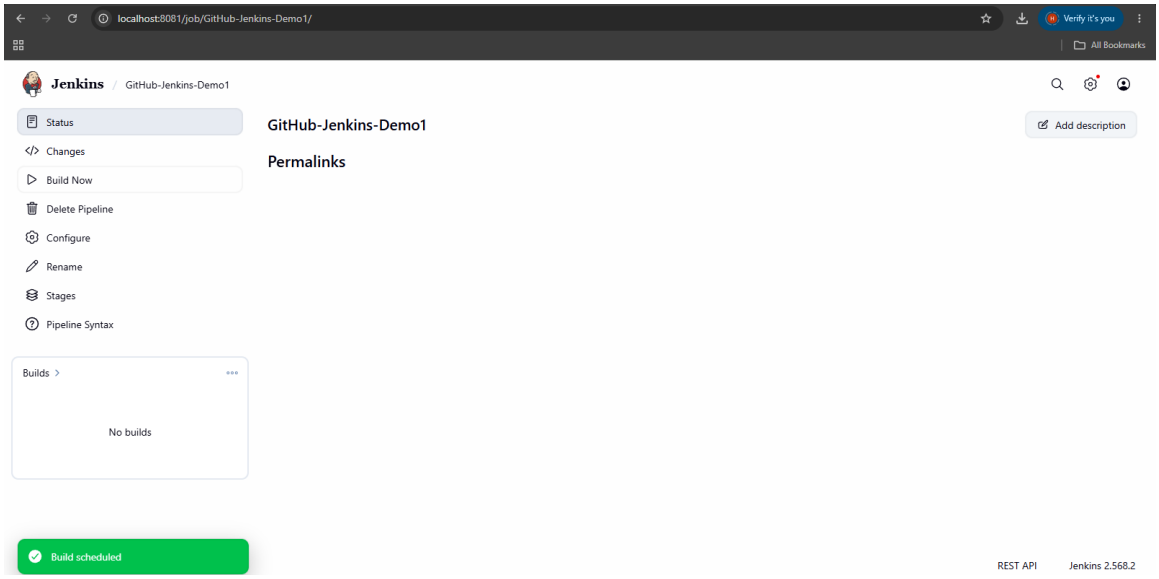


Fig 4: Build scheduled after clicking 'Build Now'

Once the build (#1) completes, it appears in the Build History. Selecting the build and choosing Console Output from its menu displays the execution log.

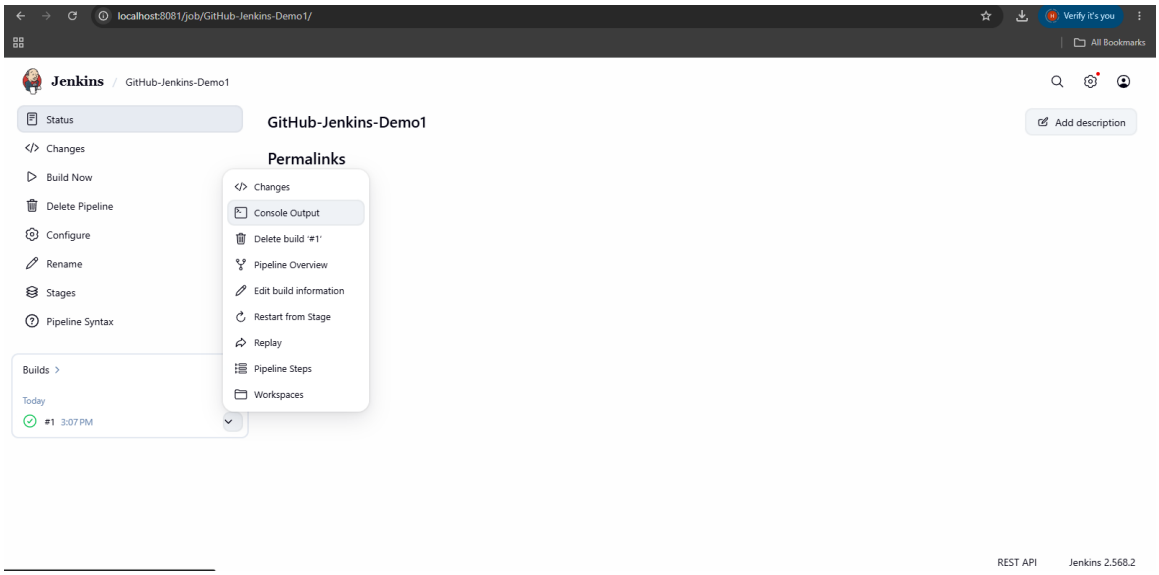


Fig 5: Build #1 completed successfully; accessing Console Output from the build menu

The console output confirms that each stage (Build, Test, Deploy) executed its echo step in sequence and that the pipeline finished with status SUCCESS, verifying that Jenkins can execute the pipeline locally.

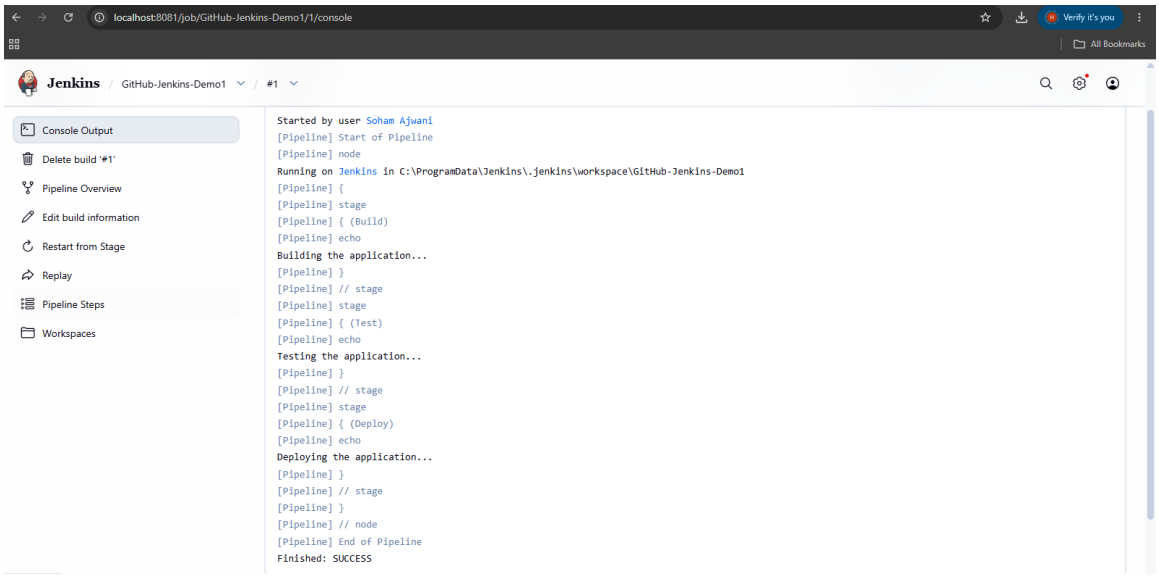


Fig 6: Console output of build #1 showing all stages executed and 'Finished: SUCCESS'

Part 4 — Preparing the GitHub Repository

A GitHub repository (github-jenkins-demo) is created and an index.html file is added, representing the sample web application that Jenkins will build, test, and deploy.

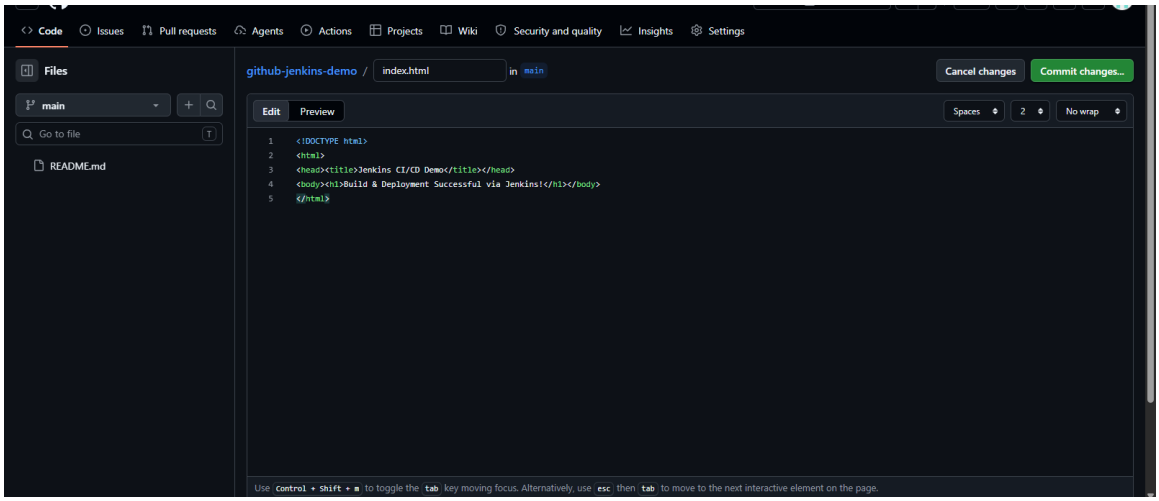


Fig 7: Adding index.html to the GitHub repository

The change is committed directly to the main branch with an appropriate commit message.

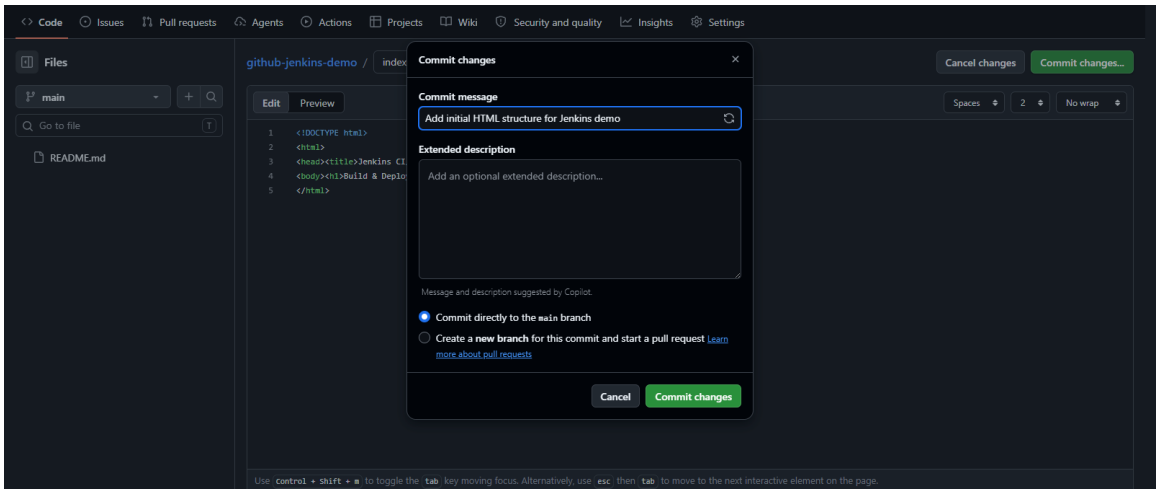


Fig 8: Committing the initial HTML structure to the main branch

A Jenkinsfile is then created in the same repository, containing the same Declarative Pipeline definition (Build, Test, Deploy stages), so that Jenkins can fetch the pipeline script directly from source control (Pipeline script from SCM).

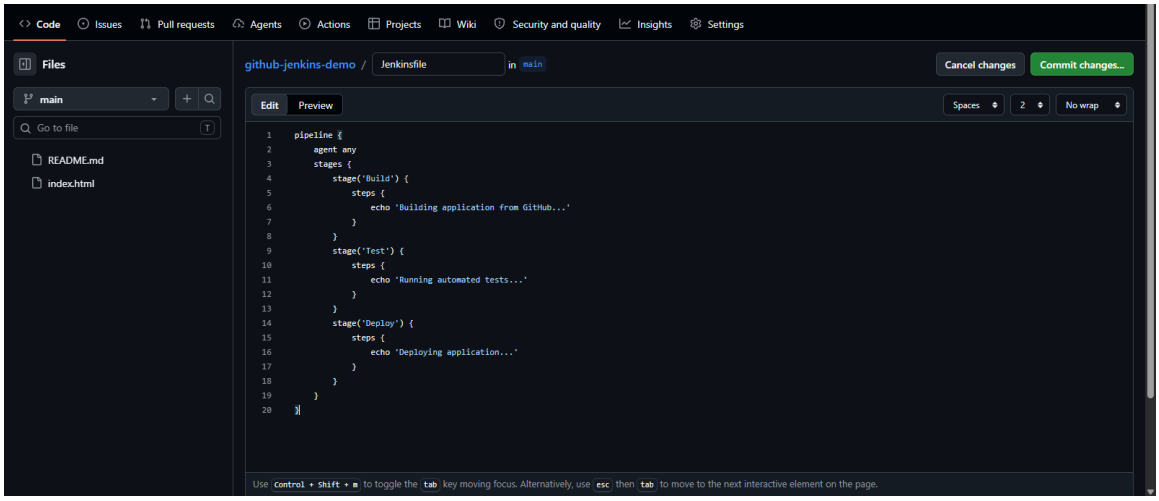


Fig 9: Creating the Jenkinsfile in the GitHub repository

The Jenkinsfile is committed to the main branch with a descriptive commit message.

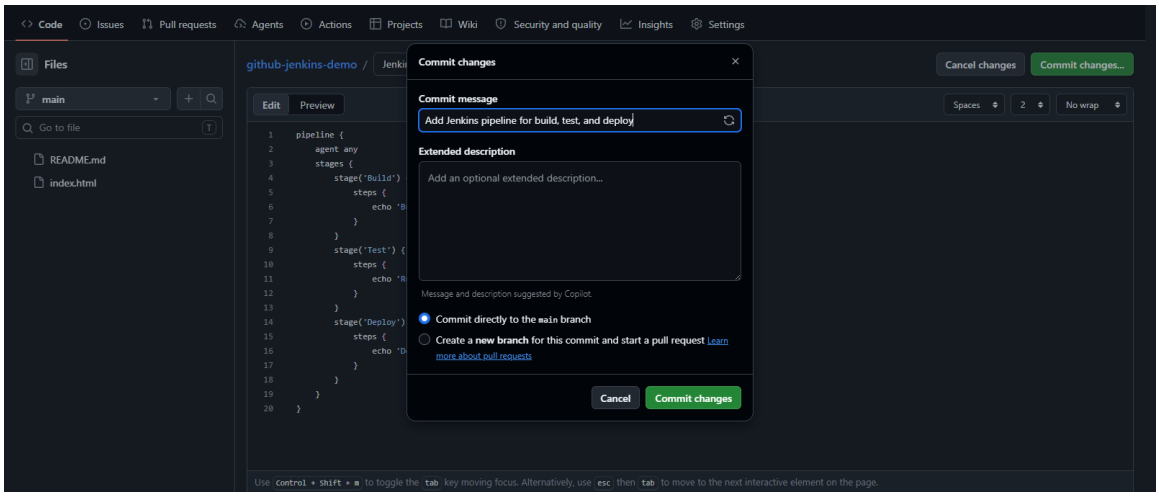


Fig 10: Committing the Jenkinsfile ('Add Jenkins pipeline for build, test, and deploy')

Part 5 — Connecting Jenkins to GitHub

In the job's Configure page, under Triggers, the 'GitHub hook trigger for GITScm polling' option is enabled so that a push to GitHub automatically triggers a Jenkins build.



Fig 11: Enabling the GitHub hook trigger under Triggers

Under the Pipeline section, the Definition is changed from Pipeline script to Pipeline script from SCM, and the Script Path is set to Jenkinsfile.

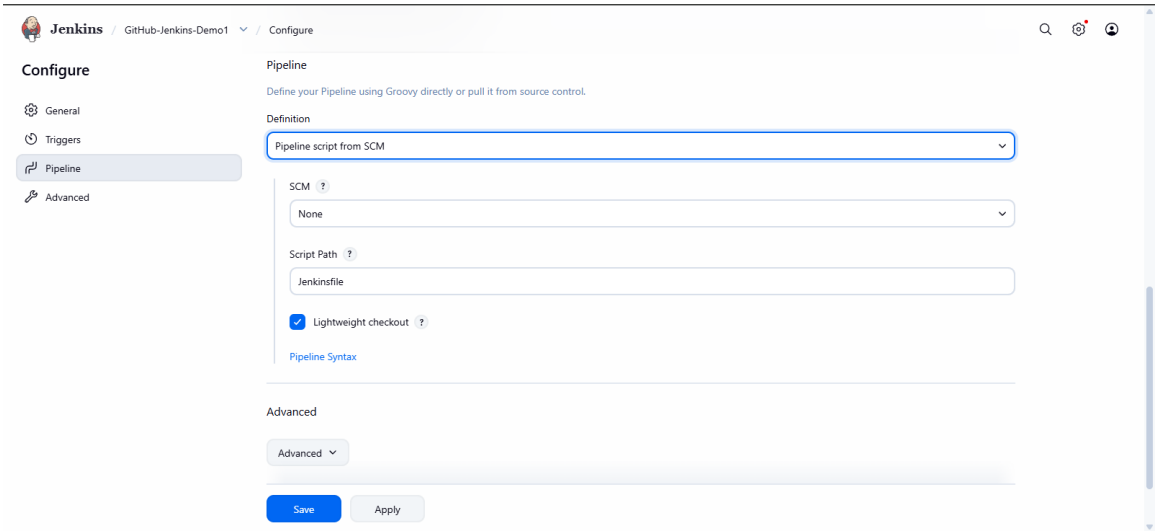


Fig 12: Setting the Pipeline definition to 'Pipeline script from SCM' with Script Path 'Jenkinsfile'

The SCM is set to Git, and the GitHub repository URL (https://github.com/sohamajwani/github-jenkins-demo.git) is entered.

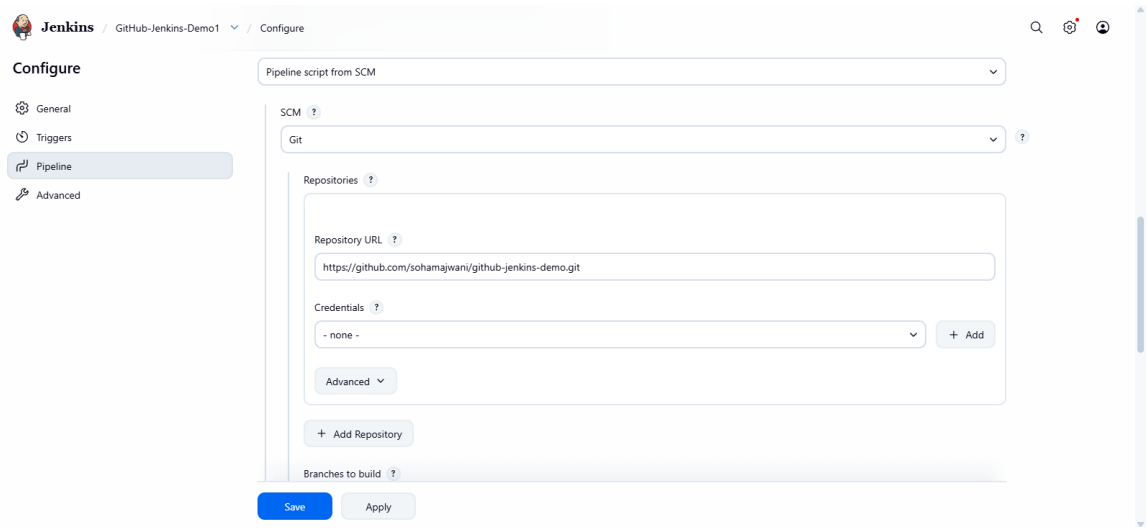


Fig 13: Configuring the Git repository URL under SCM

The Branch Specifier is set to */main and the Script Path is confirmed as Jenkinsfile, after which the configuration is saved.

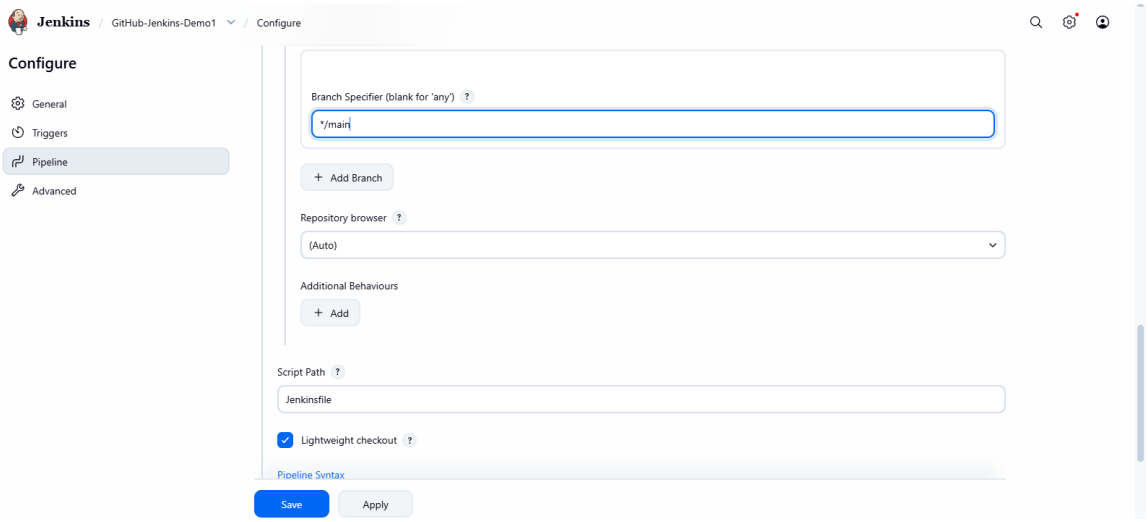


*Fig 14: Setting the branch specifier to */main and confirming the Jenkinsfile path*

Part 6 — Verifying the GitHub-Triggered Build

After saving, the pipeline status shows the last build, last stable build, and last successful build details for the job.

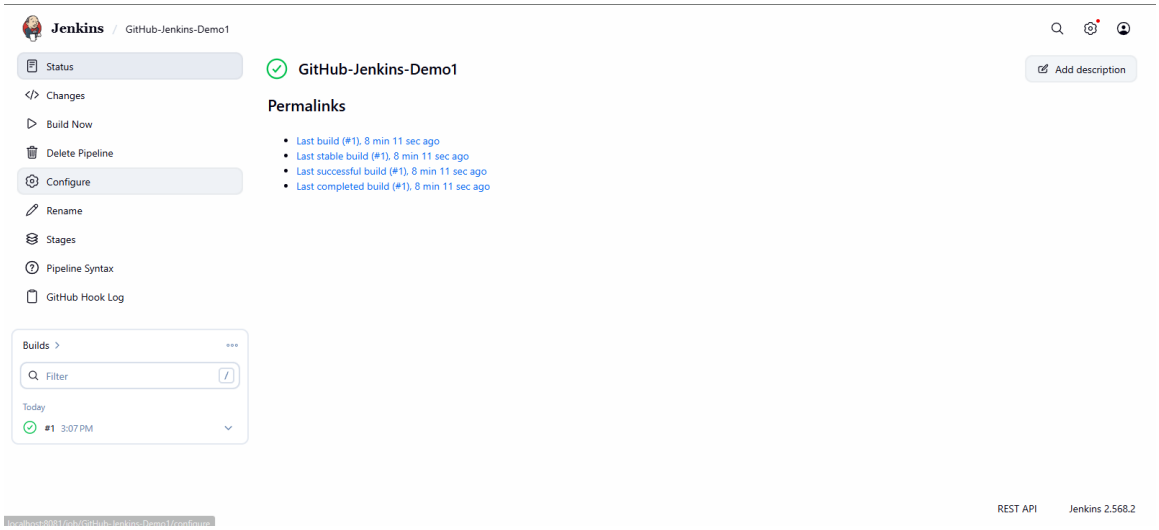


Fig 15: Job status page showing successful build history after SCM configuration

Triggering the build again schedules a new run of the pipeline sourced from GitHub.

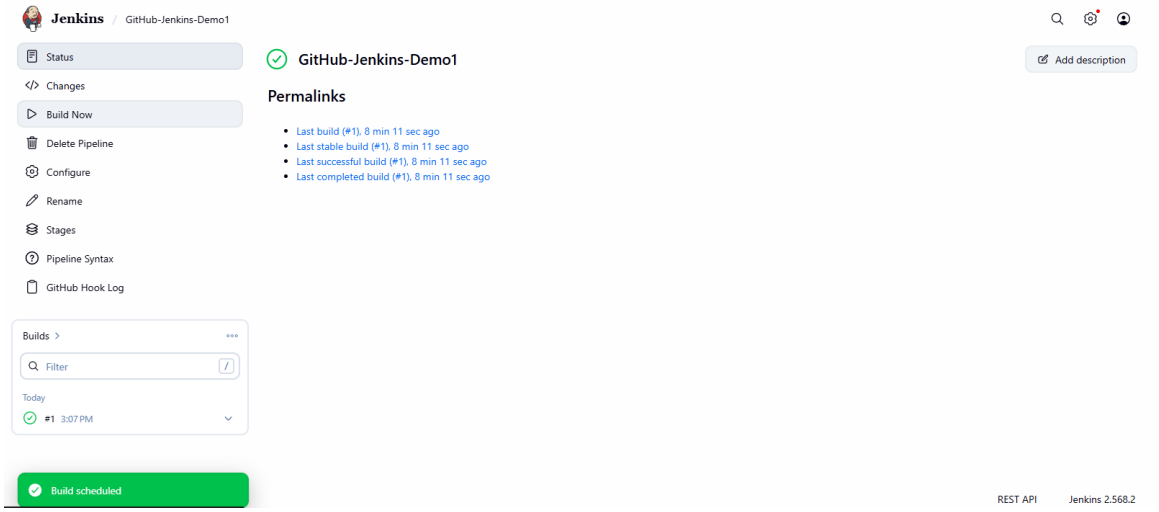


Fig 16: 'Build scheduled' notification for the GitHub-sourced pipeline

Build #2 completes successfully, and its Console Output is opened from the build menu.

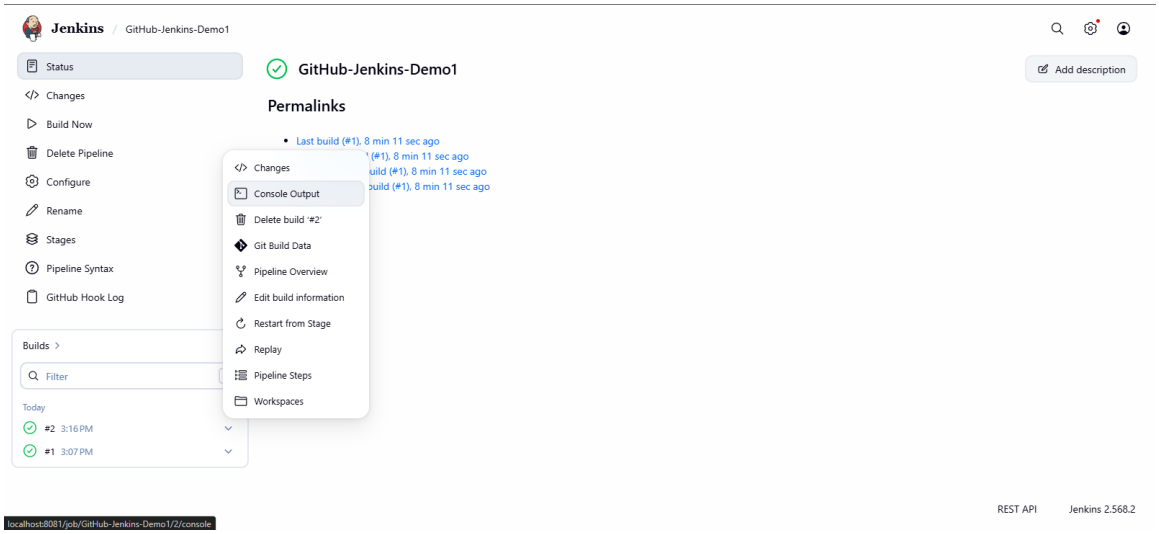


Fig 17: Build #2 completed; opening Console Output from the build menu

The console output shows Jenkins obtaining the Jenkinsfile directly from the GitHub repository and cloning it into the local workspace, confirming the SCM integration is working.

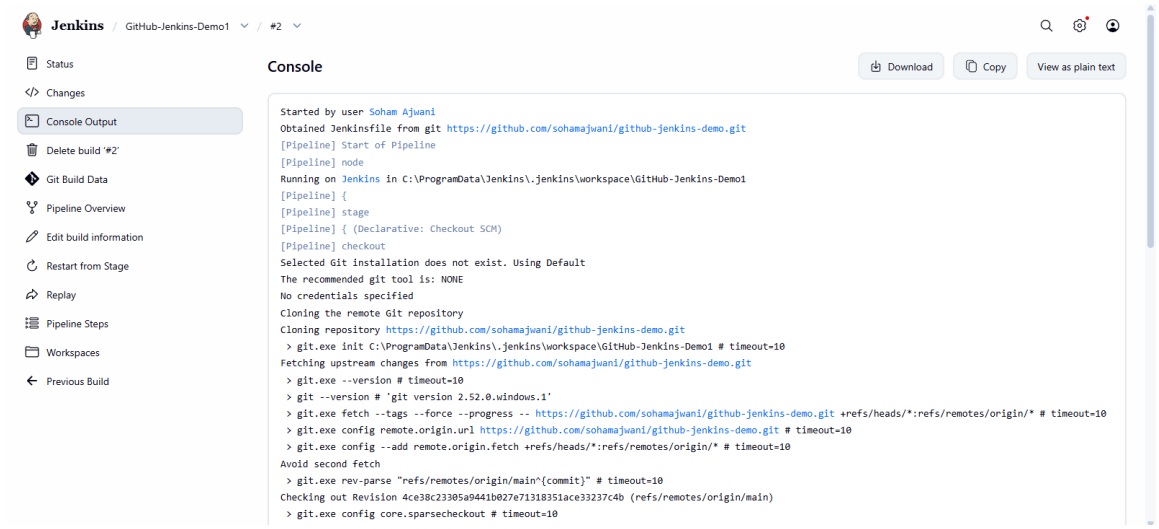


Fig 18: Console output showing Jenkins cloning the repository and obtaining the Jenkinsfile from GitHub

Jenkins checks out the latest commit from the main branch and executes the Build, Test, and Deploy stages defined in the Jenkinsfile.

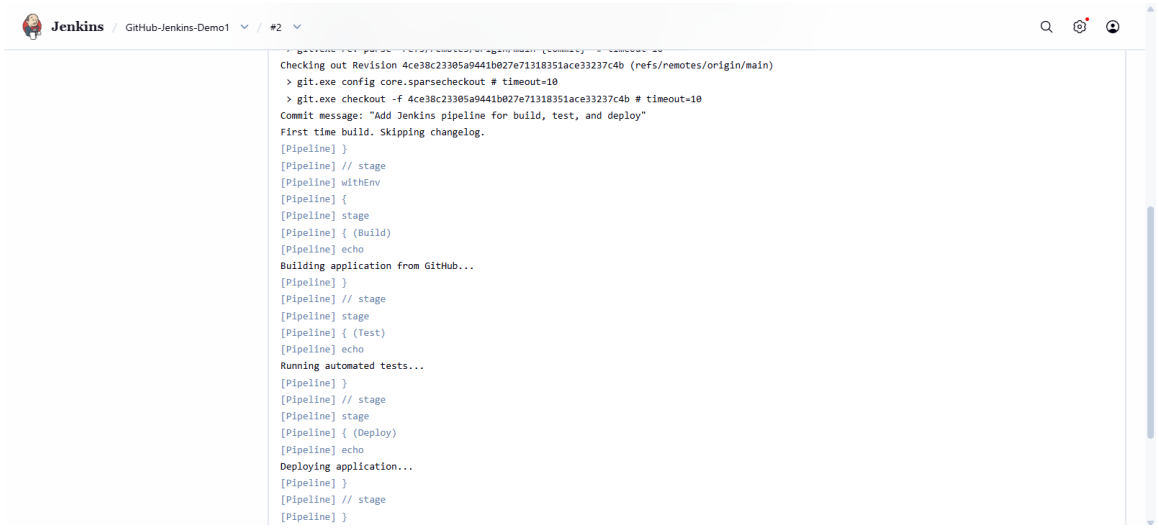


Fig 19: Checkout of the latest commit followed by execution of Build, Test, and Deploy stages

The pipeline completes all stages and finishes with status SUCCESS, confirming that the full GitHub → Jenkins CI/CD flow works end-to-end.



Fig 20: Pipeline execution completed — 'Finished: SUCCESS' for the GitHub-triggered build

Conclusion

Jenkins was successfully installed and configured, and a Declarative Pipeline consisting of Build, Test, and Deploy stages was created and executed. The pipeline was first verified locally with a manually written script, and subsequently integrated with a GitHub repository by configuring 'Pipeline script from SCM' and enabling the GitHub hook trigger. This allowed Jenkins to automatically fetch the Jenkinsfile from GitHub, check out the latest commit, and run the complete pipeline, which finished with status SUCCESS. Through this experiment, a practical understanding was gained of how Jenkins and GitHub work together to implement a basic CI/CD workflow, automating the process of building, testing, and deploying an application whenever new code is pushed to the repository.